Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Control Engineering and Information Technology

Algorithms for Real-Time Physics Simulation

Bachelor's Thesis

Author
Márton Biró

Advisor
dr. Márton Vaitkus

May 24, 2026

Contents

STUDENT DECLARATION

I, *Márton Biró*, the undersigned, hereby declare that the present Bachelor's Thesis work has been prepared by myself and without any unauthorized help or assistance. Only the specified sources (references, tools, etc.) were used. All parts taken from other sources word by word, or after rephrasing but with identical meaning, were unambiguously identified with explicit reference to the sources utilized. In the Appendix, I have clearly indicated whether I used artificial intelligence tools in preparing the thesis; if so, I have specified the manner and extent of their use in the table. I acknowledge that I bear full responsibility for any content generated by artificial intelligence – regardless of the extent of its use.

I agree that the basic information of my present work (author(s), title, abstracts in English and in a second language, year of preparation, name of advisor(s)) will be made publicly available on the internet by BME VIK, and the full text of the work will be available to persons registered in the BME Directory. I declare that the submitted hardcopy of the thesis work and its electronic version are identical.

Full text of thesis works classified upon the decision of the Dean will be published after a period of three years.

Budapest, May 24, 2026

Márton Biró
student

Kivonat

TODO

Abstract

TODO

Chapter 1

Introduction

1.1 Motivation

- Real-time simulation of deformable objects (cloth, soft bodies, hair) is a baseline expectation in games, VR, and AR.
- The simulator must deliver believable dynamics within a tight frame budget: at 60 fps a frame allows ∼16 ms for all physics, rendering, and gameplay logic, with physics getting only a fraction.
- I focus on deformable bodies rather than rigid bodies because:
  - the two studied algorithms differ most visibly on deformable material — rigid-body simulation hides the per-constraint convergence behaviour I want to measure;
  - deformable simulation is the strictly harder case (many more degrees of freedom and constraints per frame), stressing both solvers so their failure modes become legible.

The accuracy/efficiency trade-off.

- Every physics-based simulator sits on a line between *physical accuracy* and *computational efficiency*; gaining one costs the other.
- Table 1.1 sketches this spectrum, ordered from most accurate to most efficient.
- XPBD and VBD both sit near the efficient end — where real-time applications live — and the thesis asks which makes the better trade-off, and under what conditions.

Designing such a simulator is hard for three recurring reasons:

- *Stiffness.* Cloth and soft tissue have very stiff constraints (almost-inextensible fabric, near-rigid bones); stiff systems make explicit integrators unstable unless time steps become impractically small.
- *Stability.* Even implicit integrators can blow up under poorly conditioned constraints, large mass ratios between connected particles, or contact-induced velocity discontinuities.

Method	Accuracy	Cost
Newton's method (implicit Euler)	reference	offline only
Projective Dynamics	high	expensive
Vertex Block Descent (VBD)	moderate–high	real-time
Extended Position-Based Dynamics (XPBD)	moderate	real-time
Position-Based Dynamics (PBD)	low (stiffness-dep.)	cheap
Explicit Euler	poor (unstable)	cheapest

Table 1.1: Deformable-simulation methods ordered from most accurate to most efficient. XPBD and VBD — the subject of this thesis — both target the real-time regime.

- *Strict time constraints.* Methods that converge well offline (Newton's method, projective dynamics) are too expensive per frame; cheap real-time methods often trade convergence quality for predictable cost.

Within the real-time band:

- XPBD [6] has been a workhorse for years.
- VBD [1] is a recent, promising addition not yet evaluated head-to-head against XPBD.
- The two occupy opposite sides of the *primal/dual* duality identified by Macklin et al. [5]: XPBD solves for Lagrange multipliers (dual variables on constraints), VBD solves directly for vertex positions (primal variables).
- No controlled comparison between them currently exists.

1.2 Contribution

- The central contribution is, to the best of my knowledge, the *first faithful XPBD-vs-VBD comparison*:
  - both algorithms implemented in the same Unity-based codebase,
  - applied to the same mass-spring material model,
  - run under a matched compute budget,
  - evaluated against a high-iteration Newton solve serving as numerical ground-truth reference.
- This sharpens the closest prior comparison, Ma et al. [4], which contrasts *plain* PBD (not XPBD) with a non-canonical VBD implementation and reports only frame-rate scaling.

I distinguish this work by:

1. using XPBD, the constraint-stiffness-independent variant any practitioner would deploy today;

2. implementing the canonical VBD formulation, including its block-coordinate-descent interpretation;
3. matching compute budgets and material parameters rather than iteration counts (one XPBD substep is not directly comparable to one VBD iteration);
4. adding a Newton ground truth, enabling reporting of *solution error* rather than only relative performance.

1.3 The primal/dual lens

- XPBD and VBD sit on opposite sides of the primal/dual split developed in Chapter 2: XPBD updates dual variables one constraint at a time, VBD updates primal positions one vertex at a time.
- This split is the lens I use to decide what is worth measuring. A dual solver is sensitive to how information propagates along chains of coupled constraints, so mass disparities along such a chain are a natural stress; a primal solver is sensitive to the conditioning of each per-vertex block, so a wide spread of stiffnesses meeting at one vertex is a natural stress. The experiments in Chapter 6 are organised around exactly these regimes.
- The goal is to map where each solver is accurate, stable, and cheap, and where it is brittle — not to argue for the primal/dual framing itself, which I take as given from [5].

1.4 Evaluation axes

I evaluate both solvers along four axes that together capture the trade-offs a practitioner cares about:

Efficiency. Wall-clock cost per frame as a function of mesh size, substep/iteration count, and target quality.

Robustness. Behaviour under stress: large time steps, single iterations, extreme mass ratios, extreme stiffness ratios, self-contact.

Physical realism. Distance from a Newton reference, energy behaviour, damping behaviour, and agreement with analytic solutions on a harmonic oscillator and a draped cloth.

Flexibility. Ease of changing material stiffness, adding new constraint types, and reasoning about parameters.

1.5 Thesis structure

- Chapter 2: theoretical foundation — variational form of implicit Euler, mass-spring energies, the primal/dual lens that makes XPBD and VBD comparable.
- Chapter 3: reviews PBD, XPBD, VBD literature and positions this work against Ma et al. [4].

- Chapter 4: states the two algorithms in canonical form.
- Chapter 5: the Unity implementation and the design decisions that keep the comparison fair.
- Chapter 6: results along the four axes.
- Chapter 7: synthesis across axes through the primal/dual lens, threats to validity.
- Chapter 8: closing with future-work directions.

Chapter 2

Background and theoretical foundations

- This chapter establishes the mathematical scaffolding for the rest of the thesis.
- The central object is the variational form of implicit Euler, which unifies XPBD and VBD as two approximate minimisers of the same energy functional, and therefore justifies a shared Newton reference as ground truth.

2.1 Time-stepping schemes for physical simulation

- Real-time physics reduces to advancing a state $(\mathbf{x}, \mathbf{v})$ — positions and velocities of n particles — forward in time.
- Three families of approaches have been used in computer graphics:

Newtonian, force-based.

- Forces $\mathbf{f}(\mathbf{x}, \mathbf{v}, t)$ produce accelerations via $\ddot{\mathbf{x}} = \mathbf{M}^{-1}\mathbf{f}$, integrated by some scheme.

Impulse-based.

- Velocities are manipulated directly via collision or constraint impulses; popular for rigid bodies.
- Not pursued here: awkward for distributed deformation, and the primal/dual framework of Section 2.5 is the right lens for comparing XPBD and VBD.

Position-based.

- Positions are manipulated directly; velocities are recovered as finite differences. This is the lineage XPBD belongs to.

2.1.1 Explicit Euler and why it fails

The explicit (forward) Euler scheme is

$$\mathbf{v}_{t+h} = \mathbf{v}_t + h\,\mathbf{M}^{-1}\mathbf{f}(\mathbf{x}_t), \tag{2.1}$$
$$\mathbf{x}_{t+h} = \mathbf{x}_t + h\,\mathbf{v}_{t+h}. \tag{2.2}$$

- For a linear spring with stiffness k and mass m, stability requires roughly $h < 2\sqrt{m/k}$.
- For cloth, where k may exceed 10^4 N/m, this drives h down to microseconds — incompatible with a 16 ms real-time frame budget.
- Velocity Verlet and symplectic-Euler variants still retain a stability limit driven by the stiffest mode.

2.1.2 Implicit Euler

The (backward) implicit Euler scheme evaluates forces at the unknown end-of-step state:

$$\mathbf{v}_{t+h} = \mathbf{v}_t + h\,\mathbf{M}^{-1}\mathbf{f}(\mathbf{x}_{t+h}), \tag{2.3}$$
$$\mathbf{x}_{t+h} = \mathbf{x}_t + h\,\mathbf{v}_{t+h}. \tag{2.4}$$

- Unconditionally stable for linear systems, dissipative for non-linear ones.
- Cost: a non-linear root-finding problem each step.

2.2 Implicit Euler as energy minimisation

Eliminating the velocity equation gives the well-known variational form of implicit Euler: the new positions $\mathbf{x}$ minimise

$$G(\mathbf{x}) \;=\; \tfrac{1}{2h^2}\,\|\mathbf{x}-\mathbf{y}\|_{\mathbf{M}}^2 \;+\; E(\mathbf{x}), \tag{2.5}$$

where

$$\mathbf{y} \;=\; \mathbf{x}_t + h\,\mathbf{v}_t + h^2\,\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}$$

- $\mathbf{y}$ is the *inertial target* (the position reached under external forces alone).
- $\|\cdot\|_{\mathbf{M}}$ is the mass-weighted norm; $E(\mathbf{x})$ is the elastic potential energy.
- Equation (2.5) is central: **both XPBD and VBD are approximate minimisers of the same** $G(\mathbf{x})$.
- XPBD reaches a stationary point by descending on the Lagrange-dual problem; VBD reaches one by block-coordinate descent on the primal.
- This shared target licenses a Newton solve of (2.5) to serve as unambiguous ground truth in Chapter 6.

2.3 Mass-spring systems

- I use a mass-spring discretisation of cloth: vertices carry lumped masses, edges of the cloth mesh carry springs.
- For a spring between particles i and j with rest length ℓ_0 and stiffness k, the constraint function and energy are:

$$C(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\| - \ell_0, \tag{2.6}$$

$$E(\mathbf{x}_i, \mathbf{x}_j) = \tfrac{1}{2}\, k\, C^2. \tag{2.7}$$

2.3.1 Distance, stretch, and bending springs

Two spring types, both expressed through the distance-spring energy above:

- *Stretch springs* between mesh-adjacent vertices oppose in-plane deformation.
- *Bending springs* are modelled as long-range distance springs between vertices opposite across an edge (the "diagonal" of the two-triangle quad).
- Note: this is *not* a dihedral-angle constraint — I deliberately stay within a single constraint form, since the goal is a controlled comparison, not a state-of-the-art cloth model.

2.3.2 Limitations of mass-spring

- Mass-spring is cheap and easy to differentiate, but is well known to be *mesh-dependent*: effective macroscopic stiffness depends on the discretisation.
- Therefore conclusions about absolute material parameters are not transferable to FEM-based models.
- Conclusions about *relative* solver behaviour — my subject — are transferable.

2.4 Solver structure: Gauss–Seidel vs. Jacobi

- Both XPBD's constraint sweep and VBD's vertex-block sweep are Gauss–Seidel-style iterations: each local update uses the most recent neighbour values, not start-of-iteration values.
- A Jacobi variant updates all blocks from the start-of-iteration state in parallel: slower convergence in serial, but maps better to the GPU.
- Consequence (revisited in Section 7.3): both solvers are **order-dependent**.
  - Constraint-traversal order (XPBD) and vertex-traversal order (VBD) both affect convergence rate and, in pathological cases, the converged solution.
  - My experiments fix a deterministic order for reproducibility.

2.5 The primal/dual distinction

- The unifying framing is the primal/dual perspective due to Macklin et al. [5].
- Minimising $G(\mathbf{x})$ in (2.5) can be approached two ways:

Primal.

- Directly minimise over positions $\mathbf{x}$.
- A vertex-by-vertex block-coordinate descent on G is exactly VBD.

Dual.

- Reformulate the energy through compliant constraints (each spring is a soft constraint with compliance $\alpha = 1/k$) and solve for Lagrange multipliers $\boldsymbol{\lambda}$ on each constraint.
- XPBD is a Gauss–Seidel iteration on the resulting saddle-point system.

This duality is the lens used throughout the thesis to decide what to measure and to interpret what is measured. It points to a few places where the two approaches naturally differ:

- Dual variables couple constraints that share a particle, so mass disparities at a shared particle are a regime where XPBD's per-constraint updates may propagate slowly.
- The primal Hessian block at a vertex collects all spring stiffnesses meeting there, so widely varying stiffnesses produce an ill-conditioned block — a regime where VBD's per-vertex update may struggle.
- The primal Hessian contains the true elastic operator, while the constraint linearisation underlying XPBD discards second-order information, which is why per-iteration convergence under increasing material stiffness is a natural point of comparison.

These are the considerations that motivate the stress tests in Chapter 6; the experiments report what each solver actually does in these regimes rather than setting out to confirm the framing.

Chapter 3

Related work

- This chapter surveys the position-based and primal-descent lineages whose endpoints — XPBD and VBD — this thesis compares.
- It positions my work relative to the closest prior comparison.

3.1 From PBD to XPBD

- Position-Based Dynamics (PBD), introduced by Müller et al. [9], targets real-time cloth and soft-body simulation in games.
- Each iteration projects positions onto constraint manifolds (e.g. fixed edge lengths) via Gauss–Seidel sweeps, recovering velocities by finite difference.
- PBD is unconditionally stable but has a known flaw: *constraint stiffness depends on iteration count and time step.*
  - Practitioners cannot specify a Young's modulus directly; they specify a stiffness coefficient that interacts unpredictably with solver settings.
- XPBD, introduced by Macklin et al. [6], fixes this by augmenting each constraint with a compliance term $\alpha = 1/k$ and solving for the associated Lagrange multiplier λ.
- The resulting iteration solves an implicit-Euler problem with arbitrary elastic energies, and constraint stiffness becomes a true material parameter independent of iteration count.

Small steps in physics simulation.

- Macklin et al. [7] (the "Small Steps" note) later argued that XPBD converges most reliably with *many short substeps and a single iteration per substep*, rather than a single long step with many iterations.
- This is now a standard XPBD configuration and is the default I adopt in Chapter 5: it follows the recommendation and exposes XPBD's per-substep cost as the primary performance lever.

3.2 Projective dynamics and descent methods

- Projective dynamics recasts implicit Euler as a quadratic minimisation alternating between a global linear solve and local constraint projections.
- Variants and accelerations (descent methods, Chebyshev acceleration, ADMM) bridge between projective dynamics and position-based methods.
- They provide the theoretical bridge to VBD: VBD can be read as a *block-coordinate descent* on the same variational energy (2.5) that projective dynamics minimises, but with single-vertex blocks rather than a global solve.

3.3 Vertex Block Descent (VBD) and successors

- Vertex Block Descent, due to Chen et al. [1], treats each vertex as a tiny optimisation block: at each iteration, every vertex moves to the minimiser of G with all other vertices held fixed.
- This requires the local 3×3 Hessian of the energies sharing the vertex and a Newton step on that block.

Critical features of the original VBD paper:

- *Chebyshev acceleration*, an over-relaxation that significantly speeds convergence at the cost of energy guarantees.
- *Adaptive initialisation*, seeding the iteration from a blend of inertial and previous-step positions.
- *Graph-colouring parallelism*: vertices sharing no constraint can be updated independently, enabling massive parallelism.

- A follow-up, Augmented VBD (AVBD) [2], has improved stiffness-ratio handling and convergence behaviour.
- I restrict the comparison to the canonical 2024 VBD since AVBD post-dates my implementation window; I cite AVBD in Section 6.4 as the most relevant counter-evidence on the stiffness-ratio axis.

3.4 Prior comparison: Ma et al. (2024)

- The most directly comparable prior work is Ma et al. [4], who compare cloth simulation under PBD and VBD in Unity.
- Their study established that VBD scales better than PBD with node count, and identified maximum sustainable node counts at 30 fps.

My work differs in three substantial ways:

1. **XPBD, not PBD.** Plain PBD has the stiffness-coupling flaw that XPBD was designed to fix, so a PBD-vs-VBD comparison conflates algorithmic differences with this known bug. Using XPBD isolates the primal/dual distinction.

2. **Canonical VBD.** Ma et al.'s pseudocode departs in details (initialisation, Chebyshev handling) from [1]. I follow canonical VBD and disable Chebyshev / adaptive init for the core comparison, so I read off algorithm properties, not acceleration tricks.

3. **Beyond frame rate.** Ma et al. report fps and node counts. I add a Newton ground truth, an equal-wall-clock quality frontier, and targeted stress tests (mass ratio, stiffness ratio, large dt) keyed to the primal/dual distinction.

- In short, this thesis answers a question Ma et al. explicitly left as future work: how does VBD compare to *XPBD*, and at *matched quality* rather than only at matched node counts?

3.5 Reference materials

- The Physics-Based Simulation book of Li et al. [3] provided canonical derivations and pedagogical notes; their treatment of implicit Euler as variational minimisation directly informs the framing in Section 2.2.

- The *Ten Minute Physics* tutorial series [8] was used as a sanity-check reference for XPBD implementation correctness.

Chapter 4

The two algorithms

- This chapter states XPBD and VBD in the form actually implemented in Chapter 5.
- Both target the same variational problem (2.5) but reach it through opposite descent directions.
- The contrast made explicit here drives the experimental design in Chapter 6.

4.1 XPBD: dual-variable position projection

4.1.1 Derivation

- For each elastic spring, associate a constraint $C(\mathbf{x}) = 0$ at zero deformation and a compliance $\alpha = 1/k$.
- The augmented Lagrangian of (2.5) with a single substep h has stationarity conditions:

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) = h^2 \, \nabla C(\mathbf{x})^\top \, \boldsymbol{\lambda}, \tag{4.1}$$

$$C(\mathbf{x}) + \tilde{\alpha} \, \boldsymbol{\lambda} = \mathbf{0}, \tag{4.2}$$

- $\tilde{\alpha} = \alpha/h^2$ is the time-step-scaled compliance.
- XPBD solves this coupled system by a Gauss–Seidel sweep over constraints: for each constraint j, the multiplier increment is

$$\Delta\lambda_j \;=\; \frac{-C_j(\mathbf{x}) - \tilde{\alpha}_j \, \lambda_j}{\nabla C_j^\top \, \mathbf{M}^{-1} \nabla C_j + \tilde{\alpha}_j}, \tag{4.3}$$

- Corresponding position correction: $\Delta\mathbf{x} = \mathbf{M}^{-1} \nabla C_j^\top \, \Delta\lambda_j$.
- Velocities are recovered at substep end by $\mathbf{v}_{t+h} = (\mathbf{x}_{t+h} - \mathbf{x}_t)/h$.

4.1.2 Algorithm

```
def xpbd_substep(x, v, h, constraints):
    y = x + h * v + h**2 * M_inv @ f_ext
    x_new = y.copy()
    lambdas = zeros(len(constraints))
    for it in range(n_iter):
        for j, C_j in enumerate(constraints):
            dlam = (-C_j.value(x_new) - alpha_tilde[j] * lambdas[j]) / \
                   (C_j.grad(x_new) @ M_inv @ C_j.grad(x_new).T
                    + alpha_tilde[j])
            x_new += M_inv @ C_j.grad(x_new).T * dlam
            lambdas[j] += dlam
    v_new = (x_new - x) / h
    return x_new, v_new
```

Listing 4.1: XPBD substep with n_{iter} Gauss–Seidel iterations.

4.1.3 Characteristics

- *Dual-variable.* An iteration's state is encoded in the multipliers $\boldsymbol{\lambda}$; positions are derived.
- *Stiffness-by-compliance.* Material stiffness enters only through $\tilde{\alpha} = 1/(kh^2)$, so very stiff constraints naturally degenerate to projection ($\tilde{\alpha} \to 0$).
- *First-order gradient information only.* Each constraint contributes its gradient ∇C_j; no Hessian of E is built.
- *Substeps preferred over iterations.* Following the small-steps result, I use many short substeps with one inner iteration.

4.2 VBD: primal block-coordinate descent

4.2.1 Derivation

- VBD performs block-coordinate descent on G in (2.5) with one block per vertex.
- Holding all other vertices fixed, the update for vertex i is the Newton step on the local 3-vector problem:

$$\mathbf{x}_i \;\leftarrow\; \mathbf{x}_i - \mathbf{H}_i^{-1}\,\mathbf{g}_i, \tag{4.4}$$

where

$$\mathbf{g}_i = \frac{m_i}{h^2}\,(\mathbf{x}_i - \mathbf{y}_i) + \sum_{e \in \mathcal{N}(i)} \nabla_i E_e(\mathbf{x}), \tag{4.5}$$

$$\mathbf{H}_i = \frac{m_i}{h^2}\,\mathbf{I}_3 + \sum_{e \in \mathcal{N}(i)} \nabla_{ii}^2\, E_e(\mathbf{x}). \tag{4.6}$$

- For a distance-spring energy $E_e = \frac{1}{2}k\,(\|\mathbf{x}_i - \mathbf{x}_j\| - \ell_0)^2$, the per-vertex gradient and Hessian are closed-form and inexpensive.

4.2.2 Algorithm

```
def vbd_step(x, v, h):
    y = x + h * v + h**2 * M_inv @ f_ext
    x_new = y.copy()  # inertial initialisation
    for it in range(n_iter):
        for i in vertex_order:
            g_i = (m[i] / h**2) * (x_new[i] - y[i])
            H_i = (m[i] / h**2) * I3
            for e in incident_edges(i):
                g_i += spring_grad_i(x_new, e)
                H_i += spring_hess_ii(x_new, e)
            x_new[i] -= solve_3x3(H_i, g_i)
    v_new = (x_new - x) / h
    return x_new, v_new
```

Listing 4.2: VBD step with n_{iter} block iterations.

4.2.3 Characteristics

- *Primal.* An iteration's state is positions $\mathbf{x}$; no auxiliary multipliers.
- *Second-order locally.* Each block update is a Newton step on the true elastic Hessian restricted to one vertex.
- *Iterations over substeps.* VBD is normally used with one long step subdivided by many block iterations; the implicit Hessian handles stiffness without tiny h.
- *Parallel-friendly.* Vertices sharing no constraint can be updated independently (graph colouring). I disable parallelism for the core comparison; see Section 5.6.

4.3 Side-by-side: where the algorithms differ

	XPBD	VBD
Variable updated	multipliers $\boldsymbol{\lambda}$	positions $\mathbf{x}$
Order of information	gradient ∇C	gradient + Hessian of E
Block size	per-constraint	per-vertex
Stiffness handled by	compliance $\tilde{\alpha}$	implicit Hessian
Typical configuration	many substeps, 1 iteration	one step, many iterations
Parallelism strategy	constraint colouring (harder)	vertex colouring (easier)

Table 4.1: Algorithmic contrast between XPBD and VBD.

- The row of Table 4.1 that drives the rest of the thesis is the first: XPBD lives in the dual, VBD lives in the primal.
- This is the distinction the primal/dual lens (Section 1.3) builds on, and the one the experiments return to when interpreting each result.

Chapter 5

Methodology and implementation

- This chapter describes the implementation that produces the data in Chapter 6.
- More importantly, it describes the decisions that keep the XPBD-vs-VBD comparison fair.

5.1 Platform and environment

- Simulators implemented inside the Unity Editor.
- All measurements in Chapter 6 were collected in this environment:
  - CPU: *AMD Ryzen 7 5800H.*
  - RAM: *16 GB DDR4.*
  - GPU: *NVIDIA GeForce RTX 3060 Laptop GPU.*
  - OS: *Windows 11 Pro.*
  - Dev Tool: *Unity 6000.4.0f1.*

TODO: used Unity functionalities: Recorder, shader graph,

5.2 Integration architecture

- Both solvers share a common scene scaffolding.
- Unity drives a fixed-rate physics tick at $1/dt = 30$ Hz; within each tick the active solver runs its update loop — substeps and inner iterations for XPBD, block iterations for VBD — on the shared particle/spring data structures.

The scaffolding handles:

- mesh and spring topology construction at scene load,
- particle/spring storage in flat C# arrays to keep cache behaviour comparable,
- recording of per-frame timing, energies, and reference solutions,
- rendering, which runs on the Unity main thread separately from the simulator.

Overhead of the shared scaffolding.

- The scaffolding adds a small constant per-frame overhead (buffer swapping, Unity message dispatch, rendering).
- Both solvers pay this overhead identically, so it does not bias the comparison.
- I report the isolated solver time (excluding render and dispatch) separately from end-to-end frame time in Section 6.1.

5.3 How the comparison is kept fair

- A common pitfall is to set some parameter "the same" that is not actually the same physical quantity on the two sides.
- I adopt four explicit fairness controls.

5.3.1 Stiffness ↔ compliance mapping

- VBD takes a stiffness k directly.
- XPBD takes a compliance $\alpha = 1/k$, scaled by the time step to $\tilde{\alpha} = \alpha/h^2$.
- To match materials I set:

$$\alpha_{\text{XPBD}} \;=\; 1/k_{\text{VBD}}, \tag{5.1}$$

- Mapping verified in a controlled test: a single spring with known analytic period simulated under both solvers; measured period required to match to within *TODO: tolerance, e.g. 1%*.
- Verification result reported in Section 6.3.

5.3.2 Matched compute budget — and the substep/iteration caveat

- The natural unit of work differs: XPBD's primary lever is substep count, VBD's is inner iteration count.
- **A substep is not an iteration.** An XPBD substep recomputes the inertial target $\mathbf{y}$ and solves a fresh constraint system; a VBD iteration refines the same step.
- Because of this asymmetry, *wall-clock* is the primary axis of comparison throughout Chapter 6.
- I sweep substep counts for XPBD and iteration counts for VBD independently, plot quality vs. ms/frame, and read the *equal-wall-clock frontier* from the resulting curves.
- Iteration-count comparisons are reported only as secondary information.

5.3.3 Matched gravity, dt, and damping

For all experiments unless otherwise noted:

- gravity $\mathbf{g} = (0, -9.81, 0)$ m/s^2,
- simulation time step $dt = 1/30$ s,
- Rayleigh damping with matched α_{Rayleigh} and β_{Rayleigh} coefficients (see Section 5.4),
- particle mass $m_i = 1$ kg unless a mass-ratio sweep is in progress.

5.3.4 Handling VBD's accelerations for the core comparison

- The 2024 VBD paper [1] reports significant speedups from Chebyshev acceleration and adaptive initialisation.
- Adaptive init is VBD's own inertial prediction, that is also present in XPBD, so I keep it enabled.
- Chabyshev-acceleration is also part of VBD, but it confounds an apples-to-apples comparison, as XPBD could also implement the same method (essentially Anderson-accelerated XPBD).

5.4 Damping models

- Both solvers implement Rayleigh damping, but through different mechanisms.

XPBD damping (γ-formulation).

- I use the multiplier-update extension of [6]:

$$\Delta\lambda_j \;=\; \frac{-C_j - \tilde{\alpha}_j\lambda_j - \tilde{\gamma}_j\,\nabla C_j^{\top}(\mathbf{x} - \mathbf{x}_{\text{prev}})}{\nabla C_j^{\top}\mathbf{M}^{-1}\nabla C_j\,(1+\tilde{\gamma}_j) + \tilde{\alpha}_j}, \tag{5.2}$$

- $\tilde{\gamma}$ encodes the dissipation coefficient.
- This couples damping into the dual update without changing the algorithm's structure.

VBD damping (implicit Hessian).

- VBD absorbs Rayleigh damping into the local Hessian $\mathbf{H}_i \to \mathbf{H}_i + \beta\,\mathbf{H}_{i,\text{el}}$ and into the gradient via a velocity term.
- Because the local solve is a Newton step, damping is handled fully implicitly with no additional outer iteration.

- Whether the two damping models produce the same physical decay at matched coefficients is analysed in Section 6.3.5, where it doubles as a fairness check.

5.5 Self-collision handling

The two solvers implement self-collision differently, by necessity:

- *XPBD self-collision* uses non-penetration constraints projected each iteration — contact enforced as a position constraint.
- *VBD self-collision* uses an elastic penalty potential added to E, contributing a gradient and Hessian to the affected vertex blocks.

- This is a model difference, not a solver difference: a penalty model *could* be implemented in XPBD and a projection model *could* be implemented in VBD, but the natural form differs.
- Flagged here so the self-collision result in **??** is read correctly: differences there are partly attributable to the contact model, not solely to XPBD vs. VBD.

5.6 Parallelisation (forward-looking)

- All measurements in this thesis are single-threaded on CPU.
- Noted here, and revisited in Section 7.3, the two solvers have different parallelisation profiles:
  - *VBD* parallelises naturally over vertex blocks via graph colouring: vertices sharing no constraint can be updated independently within a colour class. Well-suited to GPU execution.
  - *XPBD* can be parallelised via constraint-graph colouring, but the colouring is finer-grained (per constraint, not per vertex) and the dependency structure is less regular.
- The expected result — parallelisation favours VBD more than XPBD — is plausible but unverified by my experiments, and stands as a threat to external validity for any claim about which solver is "faster overall".

Chapter 6

Experiments and results

- This chapter reports measurements along the four evaluation axes from Section 1.4: efficiency, robustness, physical realism, flexibility.
- All experiments use the shared scaffolding and fairness controls of Section 5.3: matched gravity, $dt = 1/30$ s, no damping, $\alpha_{\text{XPBD}} = 1/k_{\text{VBD}}$, Chebyshev acceleration disabled in VBD for the core comparison.

6.1 Computational efficiency

6.1.1 Influence of substep and iteration count on error and compute time

- Sweep through different substep and iteration configurations for each solver and plot their Pareto line (optimal solutions) based on avg ms/frame and avg??? Root Mean Squared position error from the Newton reference. Each diagram compares to a Newton with different substep counts
- XPBD substep count $n_{\text{sub}} \in \{1, 2, 4, 8, 16, 32\}$
- VBD iteration count $n_{\text{it}} \in \{1, 2, 4, 8, 16, 32\}$
- The simulated problem: a chain with 10 nodes, one end fixed, the other having a weight of ??? (with bending???) . Both ends start from the same height, only gravity acting on it.
- One VBD iteration apprióximates the Newton solution better than one XPBD iteration, but are also more expensive.
- From Newton with 4 substeps graph: VBD with the same substep count as the reference Newton converges really good.
- From Newton with more substeps: iterations lose their value as with small time steps the problem becomes easy enough for both solvers to converge with 1 or 2 iterations. XPBD takes over because of its cheaper iteration cost.

TODO: add 1ss, 2ss, 4ss, 64ss graphs next to each other

6.1.2 Frame time vs. node count

- Measure FPS over a hanging-cloth scene with wind interaction at mesh resolutions *TODO: list resolutions, e.g.* 8×8 *to* 128×128, holding solver settings fixed at the Section 5.3 defaults.
- Avg FPS in steady state.
- Based on the diagrams in ssec:ErrorVsMs, I choose substep 4 iteration 4 for XPBD and substep 4 iteration 2 for VBD, as these sit at the middle point of "realistic vs. efficient" and are alway close to each other

TODO: add table containing FPS values: 8x8 16x16 32x32 64x64 128x128 XPBD ss4it4 133.8 84.5 36.0 10.6 2.8 VBD ss4it2 136.0 92.4 42.8 13.6 3.7 VBD ss4it4 124.7 73.4 29.2 8.1 2.1 TODO: Add visual images next to each other. Top row: 5 XPBD pictures, bottom row: 5 VBD pictures.

6.2 Robustness and stability

6.2.1 Chain with heavy weight attached

- A chain with a heavy weight attached to the free end. The weight of its last node is 1000 times the mass of the other nodes.
- This is the mass-disparity regime the primal/dual lens flags for XPBD's per-constraint updates.

TODO: the 3 images next to each other: Newton, XPBD then VBD

6.2.2 Chain with high stiffness ratio

- A chain with alternating stiff and soft springs. The ratio of springs is 1:10000 to mirror the experiment in the original VBD paper [1].
- This is the ill-conditioned-block regime the primal/dual lens flags for VBD's per-vertex update.

TODO: 3 images next to each other: Newton, XPBD then VBD

6.3 Physical realism

6.3.1 Newton reference

- Build a high-iteration Newton solver targeting $\|\nabla G\| < 10^{-9}$ (or until further iterations reduce G by less than 10^{-12}).
- The Newton solver shares storage and energies with the production solvers, so the comparison is over algorithm, not material.
- Error metric throughout: mass-weighted position error $\|\mathbf{x} - \mathbf{x}^\star\|_{\mathbf{M}}$, where $\mathbf{x}^\star$ is the Newton solution.

6.3.2 Harmonic oscillator

- A single spring-mass system oscillating in 1D, compared against the analytic solution $x(t) = A\cos(\omega t - \phi)\, e^{-\zeta\omega t}$.
- I measure:
  - period as a function of h and of iteration / substep count,
  - amplitude decay rate as a function of damping coefficient.
- This is the analogue of Fig. 2 in [6].

TODO: Figure — measured period and decay, both solvers, against analytic.

6.3.3 Energy tracking

- For an undamped free-falling and bouncing cloth, plot total mechanical energy over time.
- Identify systematic energy gain (instability) or loss (numerical damping) for each solver.

TODO: Figure — energy vs. time, both solvers.

6.3.4 Hanging cloth: stretched area vs. reference

- A square cloth pinned at the top corners, allowed to settle.
- Measure steady-state stretched area against the Newton reference.
- A realism check at static equilibrium, where transient dynamics drop out.

TODO: Figure — stretched area error vs. stiffness.

6.3.5 Damping cross-check

- At matched Rayleigh coefficients α_{Rayleigh} and β_{Rayleigh}, measure the amplitude-decay curve produced by each solver on the harmonic oscillator.
- Identical decay confirms the damping models of Section 5.4 are calibrated consistently; this both validates fairness and is a result in its own right.

TODO: Figure — decay envelopes overlaid.

6.4 Flexibility

6.4.1 Iteration-count independence of stiffness

- For each solver, sweep stiffness k over several decades and measure how many iterations / substeps are needed to reach a fixed error threshold.

- In the compliance formulation, XPBD's per-iteration behaviour is roughly stiffness-independent (Fig. 6 in [6]).
- Figure 6.1 shows the settled shape of a pinned cloth at four iteration / substep counts for both solvers. For both methods the resting shape is essentially unchanged as the iteration count grows, so the converged configuration does not depend on how many iterations are spent.
- For both solvers the resting configuration is essentially unchanged across the sweep, indicating that the converged shape is independent of the iteration count.

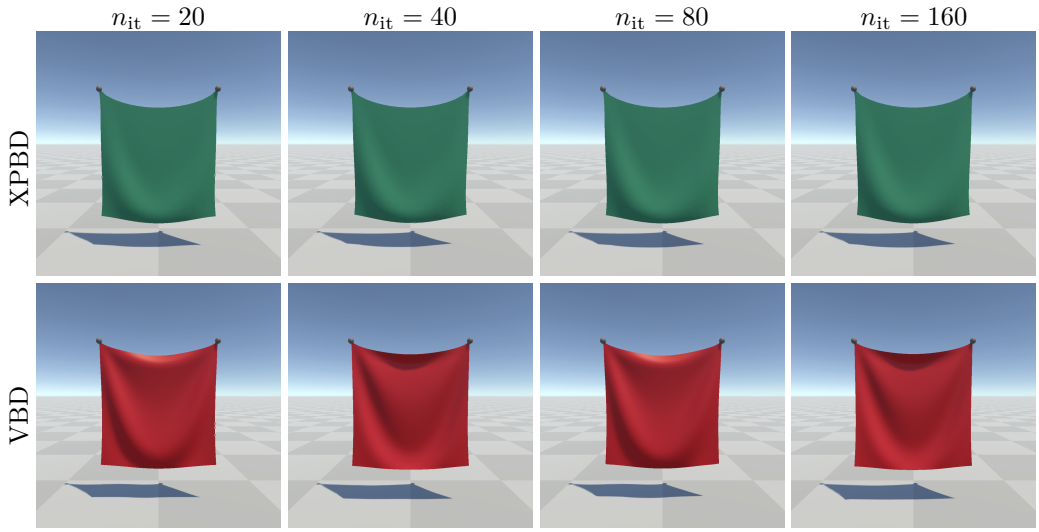

Figure 6.1: Settled shape of a pinned cloth at increasing iteration counts ($n_{\text{it}} = 20, 40, 80, 160$). Note that in this experiment the material parameters are different for the two solvers.

6.4.2 Stiffness-ratio test

- Same kind of plot as the previous subsection, but varying the *ratio* of stiffnesses across the mesh rather than a uniform stiffness.
- This is the experiment of [1] Fig. 24 (and AVBD [2] Fig. 4).
- Includes a brief discussion of AVBD's improvement on this axis without implementing it myself.

TODO: Figure — error vs. stiffness ratio, with AVBD reference qualitatively cited.

6.4.3 Ease of adding a new constraint or force

I implement a representative additional force — *TODO: choose, e.g. bending-angle constraint or area-preservation force* — in both solvers and report:

- lines of code added,

- whether a Hessian derivation was required (VBD: yes; XPBD: no, a gradient suffices),
- time to first correct simulation.

- The metric is admittedly subjective but captures a real practitioner cost that the more quantitative experiments above do not.

TODO: Table — LOC, derivation effort, time-to-correct.

Chapter 7

Discussion

- This chapter synthesises results across the four evaluation axes.
- It reads each axis through the primal/dual lens (Section 1.3) and enumerates the threats to validity that bound the conclusions.

7.1 Synthesis across axes

TODO: once experimental results are in, write a paragraph per axis that explains how the result on that axis is consistent (or not) with the primal/dual framing of Section 2.5.

- Efficiency: which solver dominates the equal-wall-clock frontier, and in what regime?
- Robustness: where do the stability envelopes diverge, and does the divergence match the primal/dual prediction?
- Realism: which solver's converged state is closer to Newton at matched compute?
- Flexibility: how does the ease-of-extension cost compare against the per-frame cost benefit?

7.2 Reading the results through the primal/dual lens

Mass-ratio regime.

- Tied to **??** and **??**.
- Interpretation: dual updates propagate slowly through chains of strongly mismatched masses, because each Gauss–Seidel sweep changes only one constraint's multiplier, and the change couples to the next constraint only through the position update on the shared particle. *TODO: state what the measurements actually show here.*

Stiffness-ratio regime.

- Tied to Section 6.2.2 and Section 6.4.2.

- Interpretation: the local Hessian block $\mathbf{H}_i$ at a vertex sums contributions from every incident spring, so widely varying stiffnesses inflate the block's condition number and can slow the Newton step. *TODO: state what the measurements actually show here.*

Per-iteration convergence under stiffness.

- Tied to **??** and Section 6.4.1.
- Interpretation: VBD uses second-order information about E locally, so increasing k need not change the number of Newton steps to convergence much, whereas XPBD discards this information. *TODO: state what the measurements actually show here.*

Joint reading.

- Read together, the three regimes describe where each solver's update direction sees the information it needs and where it does not.
- Where the results bear this out, the practical recommendation is regime-based rather than absolute — a useful outcome in its own right. *TODO: finalise once results are in.*

7.3 Threats to validity

7.3.1 Single-threaded execution

- All measurements run single-threaded on CPU.
- VBD is known to parallelise better than XPBD (Section 5.6), so ms/frame results underestimate VBD's likely production advantage.
- This affects external validity of any quantitative efficiency claim but does not invalidate qualitative claims about convergence quality or stability envelopes.

7.3.2 Mass-spring discretisation only

- I use distance-spring energies throughout, including for bending; real cloth simulators use FEM-based stretch energies and dihedral bending.
- The primal/dual framing carries over (both XPBD and VBD admit arbitrary elastic energies), but the specific stiffness ratios encountered — and therefore the magnitude of the stiffness-ratio and per-iteration-convergence effects — may differ.

7.3.3 Gauss–Seidel order dependence

- Both solvers are order-sensitive (Section 2.4).
- I fix a single deterministic order; an adversarial reordering could in principle shift results.
- I do not investigate this, and it is a real threat to reproducibility on other implementations.

7.3.4 Mismatched collision models

- XPBD uses projection-based self-collision; VBD uses penalty-based self-collision (Section 5.5).
- Any difference in the self-collision experiments mixes algorithmic and contact-model effects.

7.3.5 Fixed $dt = 1/30$ s

- $dt = 1/30$ s is generous for Unity but stresses both solvers.
- A different fixed dt, or an adaptive step, could shift the equal-wall-clock frontier.
- The stability map in **??** mitigates this by characterising both solvers over a dt range, but the headline results in Section 6.1 are tied to a single dt.

7.4 Limitations

Beyond the threats above, three further limitations are worth naming:

- I do not implement AVBD or any post-2024 VBD extension.
- Self-collision uses a simple model; production simulators often use continuous collision detection (CCD), which both solvers can be augmented with at additional cost.
- Friction is not modelled; the experiments are friction-free.

Chapter 8

Conclusion and future work

8.1 Summary

- This thesis presented a controlled comparison of XPBD and VBD, the two dominant solvers for real-time deformable simulation.
- The contribution is threefold:
  - a shared Unity codebase implementing both solvers under matched material parameters and compute budget;
  - a Newton ground-truth reference, enabling reporting of solution error rather than only relative performance;
  - an experimental design that uses the primal/dual distinction of [5] as a lens to choose which regimes to stress-test.

TODO: once results are finalised, a single paragraph summary of Section 7.2 goes here: how each solver behaves in the regimes the primal/dual lens flags, what the equal-wall-clock frontier looks like, and the regime-based recommendation for practitioners.

8.2 Future work

8.2.1 Multithreaded and GPU comparison

- The most consequential extension is to repeat the experiments on a multithreaded CPU and on the GPU.
- Both solvers admit parallelisation via graph colouring, but the structures differ in granularity (Section 5.6).
- Expected outcome: VBD's per-vertex blocks colour more cleanly than XPBD's per-constraint blocks, amplifying its production advantage; quantifying this gap is the natural next study.

8.2.2 Extend research to other dynamic types

- Beside mass-spring systems, in theory both solvers could also handle other cloth representations, soft bodies, and rigid bodies.

- It would be interesting to see how the two solvers perform on these other types of dynamics, and whether the same conclusions hold.

8.2.3 FEM constraints

- Replacing mass-spring with FEM-based stretch and bending energies would lift the realism ceiling and show whether the regime-dependent behaviour observed here survives the change of discretisation.

8.2.4 AVBD and post-2024 variants

- Augmented VBD [2] reportedly handles wide stiffness ratios substantially better than canonical VBD.
- Re-running the stiffness-ratio experiments with AVBD would clarify whether the gap I observe is intrinsic to the primal approach or specific to the canonical 2024 implementation.

Acknowledgements

Ez nem kötelező, akár törölhető is. Ha a szerző szükségét érzi, itt lehet köszönetet nyilvánítani azoknak, akik hozzájárultak munkájukkal ahhoz, hogy a hallgató a szakdolgozatban vagy diplomamunkában leírt feladatokat sikeresen elvégezze. A konzulensnek való köszönetnyilvánítás sem kötelező, a konzulensnek hivatalosan is dolga, hogy a hallgatót konzultálja.

Bibliography

[1] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. Vertex block descent. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2024)*, 43(4):116:1–116:16, 07 2024. ISSN 0730-0301. DOI: 10.1145/3658179. URL https://doi.org/10.1145/3658179.

[2] Chris Giles, Elie Diaz, and Cem Yuksel. Augmented vertex block descent. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2025)*, 44(4), 08 2025. ISSN 0730-0301. DOI: 10.1145/3731195. URL https://doi.org/10.1145/3731195.

[3] Minchen Li, Chenfanfu Jiang, Zhaofeng Luo, Wenxin Du, Chang Yu, Žiga Kovačič, and Tianyi Xie. *Physics-Based Simulation*. March 2026. URL https://phys-sim-book.github.io/.

[4] Jun Ma, Nak-Jun Sung, Min-Hyung Choi, and Min Hong. Performance comparison of vertex block descent and position based dynamics algorithms using cloth simulation in unity. *Applied Sciences*, 14(23), 2024. ISSN 2076-3417. DOI: 10.3390/app142311072. URL https://www.mdpi.com/2076-3417/14/23/11072.

[5] M. Macklin, K. Erleben, M. Müller, N. Chentanez, S. Jeschke, and T. Y. Kim. Primal/dual descent methods for dynamics. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA '20, Goslar, DEU, 2020. Eurographics Association. DOI: 10.1111/cgf.14104. URL https://doi.org/10.1111/cgf.14104.

[6] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. Xpbd: position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games*, MIG '16, page 49–54, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450345927. DOI: 10.1145/2994258.2994272. URL https://doi.org/10.1145/2994258.2994272.

[7] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. *Proceedings of the 18th annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 2019. URL https://api.semanticscholar.org/CorpusID:197928472.

[8] Matthias Müller. Ten minute physics. https://matthias-research.github.io/pages/tenMinutePhysics/index.html, 2022.

[9] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, 2007. ISSN 1047-3203. DOI: https://doi.org/10.1016/j.jvcir.2007.01.005. URL https://www.sciencedirect.com/science/article/pii/S1047320307000065.

d

Appendix

A.1 Declaration on the Use of Generative Artificial Intelligence

☐ **I have not used** any generative AI tools.

☐ **I have used generative AI tools.** I have verified the content generated by AI, ensured the accuracy of the outputs, and properly indicated each instance of use in the table below.

Usage type	**Name of Generative AI Tool(s)**	**Affected Sections** (chapter, page number, reference)	**Estimate Proportion of Use** (per usage type)
Literature Review			
Brief Summary of the Prompt			
Program Code Generation			
Brief Summary of the Prompt			
Generating New Ideas or Solution Proposals			
Brief Summary of the Prompt			
Creating an Outline (text structure, bullet points)			
Brief Summary of the Prompt			
Creating Text Blocks			
Brief Summary of the Prompt			
Generating Images for Illustrative Purposes			
Brief Summary of the Prompt			
Data Visualization, Generating Charts Based on Data Points			
Brief Summary of the Prompt			
Preparing a Presentation			
Brief Summary of the Prompt			
Other (please specify)			
Brief Summary of the Prompt			
Aggregated Percentage Value (for the core part of the task):			
Brief Textual Justification of the Aggregated Value:			